

Reporte de práctica: Inclinómetro con control de motorreductor

Práctica 3.2.3 · Sistemas Programables

1. Introducción

En esta práctica se armó un sistema que detecta qué tan inclinado está un sensor y, según ese ángulo, mueve un motorreductor. El sensor es un MPU-6050 (acelerómetro más giroscopio), el motor se maneja con un puente H L298N y todo se controla con un Arduino UNO R4 WiFi.

Lo que se mide es el cabeceo (*pitch*), es decir, el giro del sensor hacia adelante o hacia atrás. Si el ángulo es positivo el motor gira hacia adelante y si es negativo gira en reversa; mientras más inclinado esté el sensor, más rápido gira. Cerca de la posición nivelada ($\pm 5^\circ$) el motor se queda quieto y a partir de 45° trabaja a su máxima velocidad.

El programa no usa `delay()` en su ciclo principal: las tareas se reparten con `millis()` y `micros()`. Además tiene una aceleración gradual del motor y un paro de emergencia si el sensor deja de responder.

2. Objetivos

- Comunicarse con el MPU-6050 por I2C leyendo y escribiendo sus registros directamente, sin librerías para el sensor.
- Obtener un ángulo de inclinación estable uniendo acelerómetro y giroscopio mediante un filtro complementario.
- Controlar sentido y velocidad de un motor de CD con el L298N y una señal PWM.
- Aplicar zona muerta, velocidad mínima y rampa de aceleración para que el motor responda sin tirones.
- Programar un paro de seguridad con recuperación automática ante fallas del sensor.
- Mostrar la inclinación en la matriz de LEDs y el estado del sistema en el Monitor Serie.
- Ejecutar cuatro tareas periódicas con `millis()` sin bloquear el programa.

3. Material y software

Hardware

- Arduino UNO R4 WiFi (incluye matriz de LEDs de 12×8)
- Módulo GY-521 con MPU-6050
- Módulo puente H L298N
- Motorreductor de CD con llanta

- Fuente de alimentación externa para el motor
- Cables de conexión y cable USB-C

Software

- Arduino IDE
- Librerías Wire y Arduino_LED_Matrix
- Monitor Serie (115200 baudios)

4. Conexiones

El sensor se conecta al bus I2C: VCC a 5 V, GND a GND, SDA al pin A4 y SCL al pin A5. Del puente H se usan tres señales de control: ENA en D9 (PWM, define la velocidad), IN1 en D8 e IN2 en D7 (definen el sentido). Para que el Arduino pueda variar la velocidad hay que quitar el jumper de ENA del módulo.

El motor va a las salidas OUT1 y OUT2 y se alimenta con una fuente externa conectada a la terminal de +12 V del L298N, nunca con el pin de 5 V del Arduino. Las tierras de la fuente, del L298N y del Arduino deben unirse en una tierra común para que las señales de control tengan la misma referencia.

El comentario inicial del código sugiere dos conexiones extra que no aparecen en el diagrama: AD0 del sensor a GND (fija la dirección en 0x68) y una resistencia de 10 k Ω entre ENA y GND, que mantiene el motor apagado mientras el Arduino arranca o se reinicia.

Señal	Pin Arduino	Conecta a	Función
ENA	D9 (PWM)	ENA del L298N	Velocidad del motor
IN1	D8	IN1 del L298N	Sentido de giro
IN2	D7	IN2 del L298N	Sentido de giro
SDA	A4	SDA del GY-521	Datos I2C
SCL	A5	SCL del GY-521	Reloj I2C
VCC / GND	5V / GND	VCC / GND del GY-521	Alimentación del sensor
Motor	—	OUT1 y OUT2 del L298N	Alimentado por fuente externa en +12V

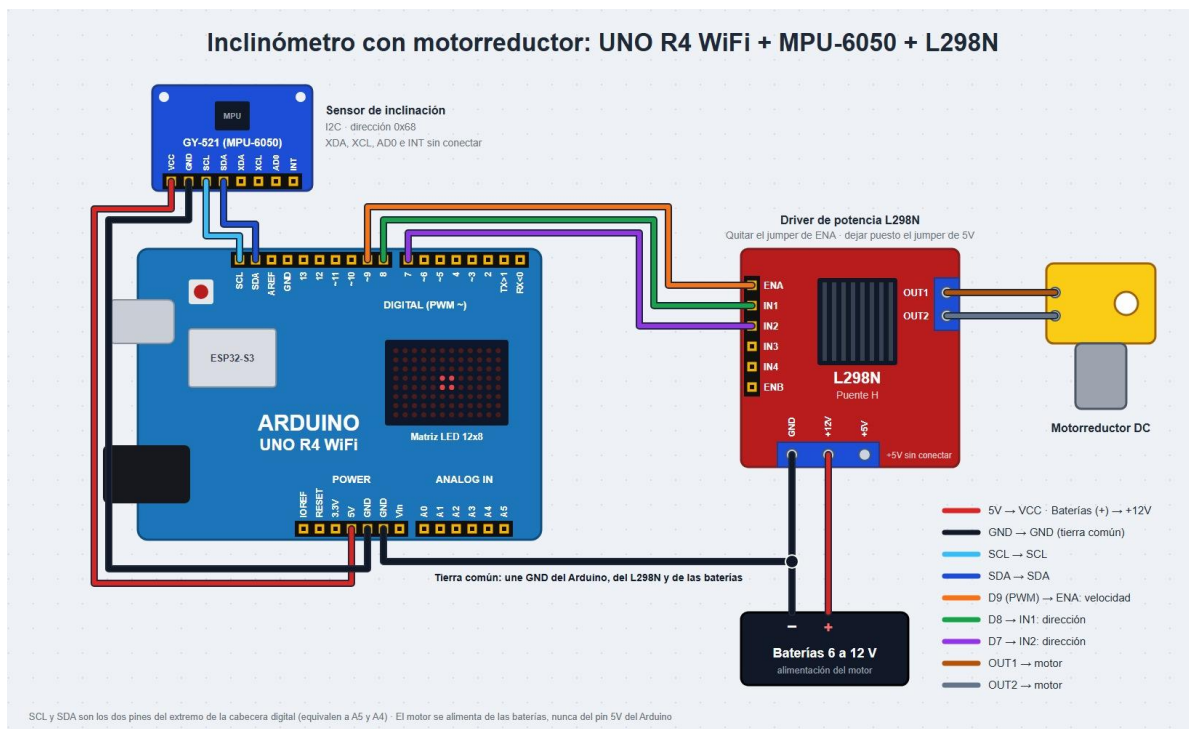


Figura 1. Diagrama de conexión del sensor, el puente H y el motorreductor con el Arduino UNO R4 WiFi.

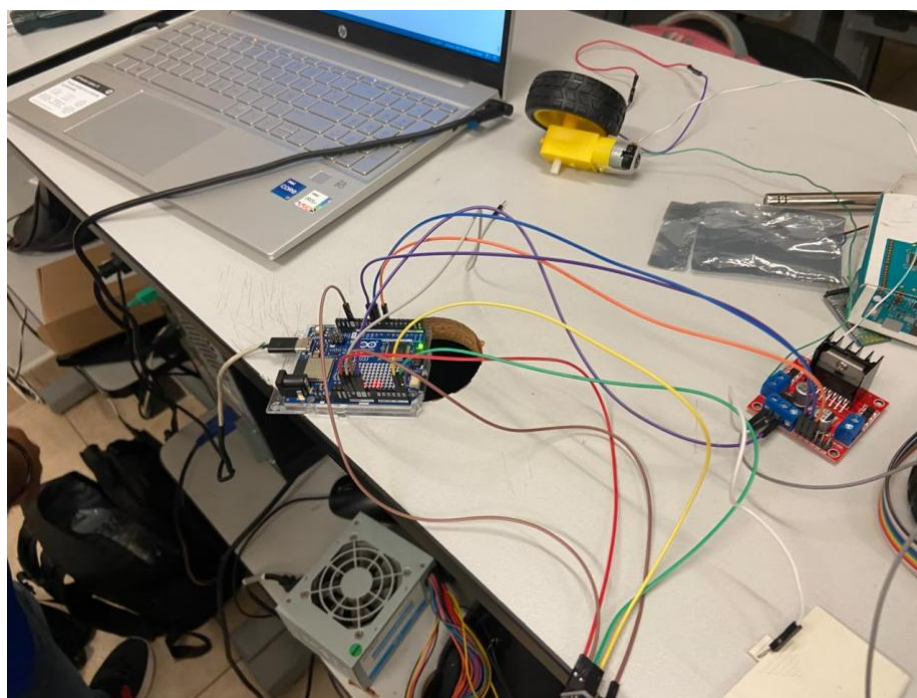


Figura 2. Montaje físico armado en el salón.

5. Desarrollo

5.1 Lectura del sensor

El MPU-6050 es un esclavo I2C en la dirección 0x68. Primero se lee el registro WHO_AM_I (0x75), que debe devolver 0x68; si no responde, el sistema queda bloqueado con el motor apagado. Después se configura: acelerómetro a ± 2 g, giroscopio a ± 250 °/s, filtro interno de unos 44 Hz y 100 muestras por segundo.

Los datos se leen en una sola transacción de 14 bytes desde el registro 0x3B. Cada valor viene en dos bytes (alto y bajo) y se convierte a unidades físicas: la aceleración se divide entre 16384 para obtener g y la velocidad angular entre 131 para obtener °/s. Antes de empezar se promedian 500 lecturas del giroscopio con el sensor quieto para obtener su desviación en reposo (*offset*), que luego se resta a cada medición.

Registros del MPU-6050 que usa el programa:

Registro	Valor escrito	Efecto
0x75 (WHO_AM_I)	— (se lee)	Debe devolver 0x68 para confirmar el sensor
0x6B (PWR_MGMT_1)	0x01	Saca al sensor del modo de reposo; reloj del giroscopio X
0x6C (PWR_MGMT_2)	0x00	Habilita acelerómetro y giroscopio
0x1B (GYRO_CONFIG)	0x00	Rango del giroscopio ± 250 °/s (131 LSB por °/s)
0x1C (ACCEL_CONFIG)	0x00	Rango del acelerómetro ± 2 g (16384 LSB por g)
0x1A (CONFIG)	0x03	Filtro pasa-bajas interno de unos 44 Hz
0x19 (SMPLRT_DIV)	0x09	$1000/(1+9) = 100$ muestras por segundo
0x3B (ACCEL_XOUT_H)	— (se lee)	Inicio de los 14 bytes: aceleración, temperatura y giro

5.2 Cálculo del ángulo

Con el acelerómetro el ángulo sale de la dirección de la gravedad: $\text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$. Es correcto cuando el sensor está quieto, pero cualquier golpe o vibración se interpreta como inclinación. Con el giroscopio se integra la velocidad de giro en el tiempo: la respuesta es suave, pero los pequeños errores se acumulan y el ángulo se va corriendo (deriva).

El filtro complementario aprovecha lo mejor de cada uno: $\text{angulo} = \alpha \cdot (\text{angulo} + \text{giro} \cdot dt) + (1-\alpha) \cdot \text{angulo_acelerómetro}$. Aquí $\alpha = \tau/(\tau+dt)$ con $\tau = 0.5$ s, que a 100 muestras por segundo da aproximadamente 0.98. Así el giroscopio domina en el corto plazo y el acelerómetro corrige la deriva poco a poco.

5.3 De ángulo a velocidad del motor

La función `calcularObjetivo()` sigue estas reglas:

- Si $|\text{ángulo}| < 5^\circ$, el PWM objetivo es 0 (zona muerta).
- Entre 5° y 45° , el PWM crece linealmente de 90 a 255.
- De 45° en adelante, el PWM se queda en 255.
- El signo del ángulo se conserva: positivo es adelante y negativo es reversa.

El valor mínimo de 90 existe porque con PWM más bajo el motor zumba pero no logra vencer la fricción de los engranes. La zona muerta evita que el motor cambie de sentido con cualquier variación de décimas de grado.

El L298N invierte la polaridad del motor: IN1 alto e IN2 bajo es adelante, IN1 bajo e IN2 alto es reversa y ambos bajos lo detienen. La velocidad la define el PWM en ENA (0 a 255).

Constantes principales del programa:

Constante	Valor	Para qué sirve
ZONA_MUERTA	5°	Por debajo de este ángulo el motor no se mueve
ZONA_CENTRO	2°	Dentro de este rango se muestra «centrado» y el marco en la matriz
ANGULO_MAXIMO	45°	Desde aquí el PWM ya es el máximo
PWM_MINIMO / PWM_MAXIMO	90 / 255	Velocidad mínima útil y máxima
PASO_RAMPA	9	Cambio de PWM cada 20 ms (29 pasos de 0 a 255)
TAU_FILTRO	0.5 s	Constante de tiempo del filtro; $\alpha \approx 0.98$ a 100 Hz
PERIODO_SENSOR / MOTOR / MATRIZ / SERIE	10 / 20 / 50 / 500 ms	Periodo de cada tarea
PERIODO_REINTENTO	250 ms	Cada cuánto se intenta recuperar el sensor en paro
ESPERA_DESPERTAR	100 ms	Espera tras reconfigurar el sensor antes de volver a ACTIVO
TIMEOUT_I2C_US	5000 μ s	Tiempo límite de una transacción I2C
MUESTRAS_CALIBRACION	500	Lecturas promediadas para el offset del giroscopio (~5 s)

5.4 Rampa de aceleración

El motor no salta directo al valor calculado. Cada 20 ms la función `actualizarRampa()` mueve el PWM aplicado hacia el objetivo en pasos de 9 unidades, así que ir de detenido a máxima velocidad toma unos 0.58 s. Si el objetivo cambia de signo, el motor primero baja hasta cero y después acelera en el otro sentido.

5.5 Estados y paro de seguridad

El programa maneja cuatro estados: BLOQUEADO (falla al arrancar), ACTIVO (operación normal), PARO y RECUPERANDO. Si una lectura del sensor falla, o si pasan más de 0.1 s entre dos lecturas, se llama a `activarParo()`: el motor se detiene al instante, sin rampa, y se avisa en el Monitor Serie. Estando en PARO, cada 250 ms se reinicia el bus I2C y se intenta identificar y configurar el sensor otra vez; si lo logra, espera 100 ms, toma el ángulo del acelerómetro como punto de partida y regresa a ACTIVO sin perder la calibración. El bus I2C tiene además un tiempo límite de 5 ms para que un cable suelto no congele el programa.

Estado	Cuándo ocurre	Qué hace el sistema
BLOQUEADO	El sensor no responde o falla la calibración al arrancar	Motor apagado, X en la matriz; hay que revisar conexiones y pulsar RESET
ACTIVO	Operación normal	Lee el sensor, calcula el ángulo y mueve el motor con rampa
PARO	Falla una lectura o el intervalo supera 0.1 s	Motor detenido al instante; reintenta el sensor cada 250 ms
RECUPERANDO	El sensor volvió a responder y se reconfiguró	Espera 100 ms y toma un ángulo inicial; luego pasa a ACTIVO

5.6 Tareas periódicas e indicadores

En cada vuelta de `loop()` se compara `millis()` contra la última ejecución de cada tarea y solo corre la que ya cumplió su periodo:

- Sensor y filtro: cada 10 ms.
- Rampa y motor: cada 20 ms.
- Matriz de LEDs: cada 50 ms.
- Monitor Serie: cada 500 ms (solo imprime cuando algo cambia).

La matriz muestra el marco completo cuando el sensor está dentro de $\pm 2^\circ$, un punto de 2×2 LEDs que sube o baja con el ángulo (hasta el borde a 45°) y una X cuando el sistema no está activo. El Monitor Serie indica dirección de la inclinación, grados, intensidad (leve $< 15^\circ$, moderada $< 30^\circ$, fuerte) y sentido y porcentaje de PWM del motor.

5.7 Fragmentos principales del código

```
float alfa = TAU_FILTRO / (TAU_FILTRO + dt);
pitch = alfa * (pitch + velocidadGy * dt)
        + (1.0f - alfa) * anguloAccel;
pwmObjetivo = calcularObjetivo(pitch);

int calcularObjetivo(float angulo) {
    const float magnitud = fabsf(angulo);
    if (magnitud < ZONA_MUERTA) return 0;
    const float limitado = magnitud > ANGULO_MAXIMO ? ANGULO_MAXIMO : magnitud;
    const float proporcional = (limitado - ZONA_MUERTA) /
```

```

        (ANGULO_MAXIMO - ZONA_MUERTA);
const int pwm = (int)(PWM_MINIMO + proporcional *
        (PWM_MAXIMO - PWM_MINIMO) + 0.5f);
return angulo >= 0.0f ? pwm : -pwm;
}

```

6. Resultados

Durante la prueba en el salón se inclinó el sensor hacia adelante y hacia atrás mientras se observaba el Monitor Serie, el motor y la matriz de LEDs.

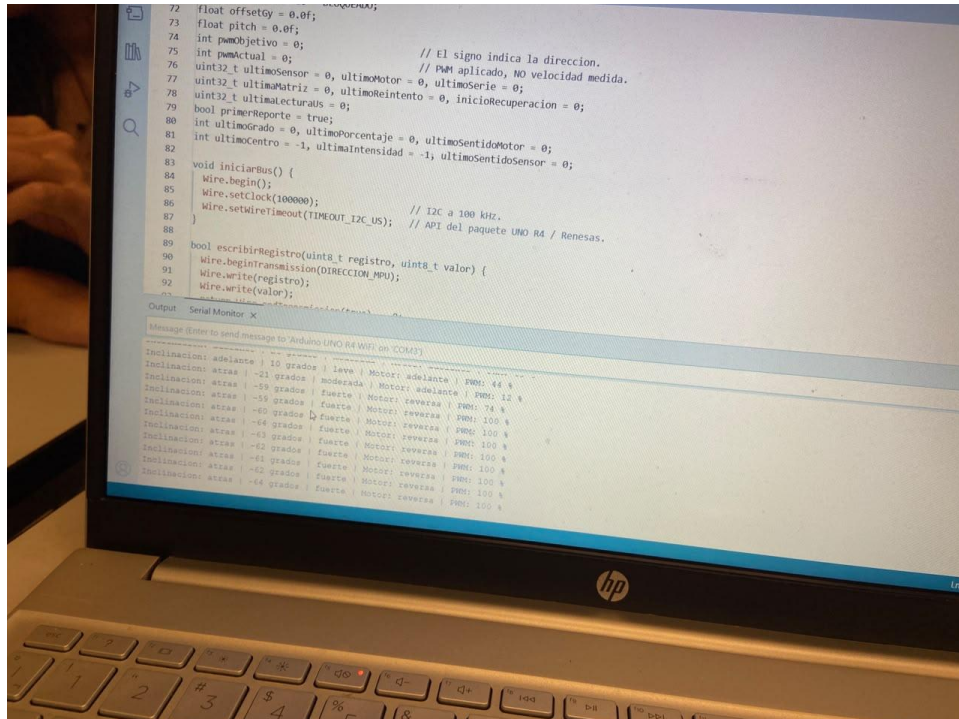


Figura 3. Monitor Serie con la inclinación y el estado del motor.

Inclinación pequeña. Con 10° hacia adelante (intensidad leve) el motor giró hacia adelante con 44 % de PWM. Coincide con el cálculo: $(10 - 5)/(45 - 5) = 0.125$ y $90 + 0.125 \times 165 \approx 111$, que es 44 % de 255.

Efecto de la rampa. Al inclinar el sensor hacia atrás, en la lectura de -21° el motor todavía giraba hacia adelante con solo 12 %: estaba frenando antes de invertir el giro. En la lectura siguiente, con -59°, ya iba en reversa al 74 %, y después llegó a 100 %.

Saturación. Entre -59° y -64° (intensidad fuerte) el PWM se mantuvo en 100 %, lo que confirma que pasados los 45° la velocidad ya no aumenta.

Tareas independientes. La matriz de LEDs mostró el indicador de inclinación mientras el Monitor Serie seguía imprimiendo, es decir, ninguna tarea bloqueó a las otras.

7. Preguntas de análisis

1. ¿Por qué no basta el acelerómetro para medir la inclinación?

Porque no distingue la gravedad de otras aceleraciones. Si el sensor se sacude o vibra, esa aceleración se suma y el ángulo calculado salta aunque no haya inclinación real. Si el motor se controlara solo con este dato, cada vibración, incluida la del propio motor, lo haría cambiar de velocidad.

2. ¿Qué pasaría con solo el giroscopio durante varios minutos?

Mide velocidad de giro, y el ángulo se obtiene sumándola. Cada lectura trae un pequeño error que también se suma, así que el ángulo se desvía poco a poco. Tras varios minutos marcaría una inclinación con el sensor nivelado y el motor empezaría a girar solo. La calibración lo reduce pero no lo elimina; por eso se corrige con el acelerómetro.

3. ¿Para qué sirve la zona muerta de $\pm 5^\circ$? ¿Y si fuera de 0° ?

Mantiene el motor detenido cerca de la posición nivelada e ignora el ruido del sensor y el temblor de la mano. Con 0° el ángulo casi nunca vale exactamente cero, así que el motor alternaría constantemente entre adelante y reversa, con vibración, calentamiento, desgaste y gasto de energía.

4. ¿Por qué la velocidad mínima es 90 y no 0?

Con un PWM bajo el motor no vence la fricción de los engranes: zumba y consume corriente sin girar. Si empezara en 0 habría un tramo de inclinación, justo después de la zona muerta, donde se ordena moverse y no pasa nada. Empezando en 90 (cerca de 35 %) gira en cuanto sale de la zona muerta.

5. Dos problemas que evita la rampa.

Primero, los picos de corriente: arrancar o invertir de golpe exige una corriente muy alta que puede bajar el voltaje, reiniciar el microcontrolador o dañar baterías y driver. Segundo, el esfuerzo mecánico: un cambio brusco, sobre todo de adelante a reversa, golpea los dientes de los engranes y los desgasta.

6. ¿Por qué el paro de seguridad es inmediato y sin rampa?

Cuando el sensor falla el sistema ya no sabe la inclinación real, y una rampa dejaría el motor girando unos 0.5 s más con un dato que ya no es confiable. Si no se sabe lo que pasa, lo seguro es detener el actuador de inmediato; la rampa protege al motor en operación normal, pero en una falla manda la seguridad.

7. ¿Por qué no se usa delay() y qué pasaría si se usara?

Porque delay() detiene todo el programa y aquí hay cuatro tareas con periodos distintos. Con un delay(500) para el Monitor Serie, el sensor se leería 2 veces por segundo en lugar de 100 y el filtro calcularía mal el ángulo, la rampa avanzaría un paso cada medio segundo (el motor tardaría unos 15 s en llegar a su máxima velocidad) y el paro de seguridad podría tardar hasta medio segundo en actuar.

8. Tres aplicaciones reales.

Vehículos autobalanceados (patinetas eléctricas, Segway), estabilizadores de cámara y plataformas niveladoras, y controles por inclinación como mandos de videojuegos o sillas de ruedas que se manejan inclinando un control.

8. Conclusión

Con esta práctica entendí cómo se integra un sistema embebido completo: un sensor que se lee directamente por I2C, un cálculo que convierte esas lecturas en un ángulo confiable y un actuador que responde a ese ángulo. Lo que más me sirvió fue ver que cada parte del programa resuelve un problema concreto: el filtro complementario corrige el ruido del acelerómetro y la deriva del giroscopio, la zona muerta evita que el motor vibre con variaciones mínimas, y el paro de seguridad detiene todo cuando el dato deja de ser confiable. También comprendí que organizar las tareas con millis() permite que el sensor, el motor, la matriz de LEDs y el Monitor Serie trabajen cada uno a su ritmo sin bloquearse.

Anexo. Código completo del programa

Programa cargado en el Arduino UNO R4 WiFi desde el Arduino IDE.

```
/*
  Practica 3.2.3: Inclinometro con control de motorreductor.
  Arduino UNO R4 WiFi + MPU-6050 + modulo puente H L298N.

  Motor: ENA=D9 (sin jumper ENA), IN1=D8, IN2=D7.
  Sensor: bus Wire, SDA=A4, SCL=A5; AD0 a GND (direccion 0x68).
  Poner una resistencia de 10 kohm entre ENA y GND: mantiene el motor
  deshabilitado mientras el Arduino arranca o se reinicia.

  Mantener el sensor QUIETO durante las 500 muestras de calibracion.
  No utiliza WiFi ni necesita una biblioteca externa para el MPU-6050.
*/

#include <Arduino.h>
#include <Wire.h>
#include <Arduino_LED_Matrix.h>
#include <math.h>
#include <string.h>

const uint8_t PIN_ENA = 9;
const uint8_t PIN_IN1 = 8;
const uint8_t PIN_IN2 = 7;
const uint8_t DIRECCION_MPU = 0x68;
const float ZONA_MUERTA = 5.0f;
const float ZONA_CENTRO = 2.0f;
const float ANGULO_MAXIMO = 45.0f;
const int PWM_MINIMO = 90;
const int PWM_MAXIMO = 255;
const int PASO_RAMPA = 9;           // 29 pasos x 20 ms = 580 ms.
const float TAU_FILTRO = 0.50f;     // Alfa aproximado: 0.98 a 100 Hz.
const float SENTIDO_PITCH = 1.0f;   // Cambiar a -1.0f si queda invertido.
const uint32_t PERIODO_SENSOR = 10;
const uint32_t PERIODO_MOTOR = 20;
const uint32_t PERIODO_SERIE = 500;
const uint32_t PERIODO_MATRIZ = 50;
const uint32_t PERIODO_REINTENTO = 250;
const uint32_t ESPERA_DESPERTAR = 100;
const unsigned int TIMEOUT_I2C_US = 5000;
const uint16_t MUESTRAS_CALIBRACION = 500;

struct LecturaIMU {
  float ax, ay, az; // Aceleracion en g.
  float gy;         // Giro sobre Y, en grados/segundo.
};

enum EstadoSistema { BLOQUEADO, ACTIVO, PARO, RECUPERANDO };

// Prototipos explicitos: tambien permiten usar LecturaIMU en el IDE Arduino.
bool escribirRegistro(uint8_t registro, uint8_t valor);
bool leerRegistros(uint8_t registro, uint8_t *datos, size_t cantidad);
bool identificarIMU();
bool configurarIMU();
bool leerIMU(LecturaIMU &lectura);
float pitchAcelerometro(const LecturaIMU &lectura);
bool calibrarGiroscopio();
void iniciarBus();
void detenerMotor();
void aplicarMotor();
int calcularObjetivo(float angulo);
void actualizarRampa();
void activarParo(const char *motivo);
void actualizarSensor();
void intentarRecuperacion(uint32_t ahora);
void informarCambios();
void actualizarMatriz();

ArduinoLEDMatrix matriz;
EstadoSistema estado = BLOQUEADO;
```

```

float offsetGy = 0.0f;
float pitch = 0.0f;
int pwmObjetivo = 0; // El signo indica la direccion.
int pwmActual = 0; // PWM aplicado, NO velocidad medida.
uint32_t ultimoSensor = 0, ultimoMotor = 0, ultimoSerie = 0;
uint32_t ultimaMatriz = 0, ultimoReintento = 0, inicioRecuperacion = 0;
uint32_t ultimaLecturaUs = 0;
bool primerReporte = true;
int ultimoGrado = 0, ultimoPorcentaje = 0, ultimoSentidoMotor = 0;
int ultimoCentro = -1, ultimaIntensidad = -1, ultimoSentidoSensor = 0;

void iniciarBus() {
    Wire.begin();
    Wire.setClock(100000); // I2C a 100 kHz.
    Wire.setWireTimeout(TIMEOUT_I2C_US); // API del paquete UNO R4 / Renesas.
}

bool escribirRegistro(uint8_t registro, uint8_t valor) {
    Wire.beginTransmission(DIRECCION_MPU);
    Wire.write(registro);
    Wire.write(valor);
    return Wire.endTransmission(true) == 0;
}

bool leerRegistros(uint8_t registro, uint8_t *datos, size_t cantidad) {
    Wire.beginTransmission(DIRECCION_MPU);
    Wire.write(registro);
    if (Wire.endTransmission(false) != 0) return false;
    if (Wire.requestFrom(DIRECCION_MPU, cantidad, true) != cantidad) return false;
    for (size_t i = 0; i < cantidad; ++i) {
        if (!Wire.available()) return false;
        datos[i] = (uint8_t)Wire.read();
    }
    return true;
}

bool identificarIMU() {
    uint8_t identificacion = 0;
    return leerRegistros(0x75, &identificacion, 1) && identificacion == 0x68;
}

bool configurarIMU() {
    return escribirRegistro(0x6B, 0x01) && // Despertar, reloj PLL giroscopio X.
           escribirRegistro(0x6C, 0x00) && // Habilitar acelerometro y giroscopio.
           escribirRegistro(0x1B, 0x00) && // Giroscopio: +/-250 grados/s.
           escribirRegistro(0x1C, 0x00) && // Acelerometro: +/-2 g.
           escribirRegistro(0x1A, 0x03) && // Filtro interno: aproximadamente 44 Hz.
           escribirRegistro(0x19, 0x09); // 1000/(1+9) = 100 muestras/s.
}

bool leerIMU(LecturaIMU &lectura) {
    uint8_t datos[14];
    if (!leerRegistros(0x3B, datos, sizeof(datos))) return false;
    const int16_t ax = (int16_t)((uint16_t)datos[0] << 8 | datos[1]);
    const int16_t ay = (int16_t)((uint16_t)datos[2] << 8 | datos[3]);
    const int16_t az = (int16_t)((uint16_t)datos[4] << 8 | datos[5]);
    const int16_t gy = (int16_t)((uint16_t)datos[10] << 8 | datos[11]);
    lectura.ax = ax / 16384.0f;
    lectura.ay = ay / 16384.0f;
    lectura.az = az / 16384.0f;
    lectura.gy = gy / 131.0f;
    // Rechazar un vector nulo: no permite conocer la inclinacion.
    const float norma2 = lectura.ax * lectura.ax + lectura.ay * lectura.ay +
                        lectura.az * lectura.az;
    return norma2 > 0.000001f;
}

float pitchAcelerometro(const LecturaIMU &lectura) {
    const float vertical = sqrtf(lectura.ay * lectura.ay + lectura.az * lectura.az);
    return SENTIDO_PITCH * atan2f(-lectura.ax, vertical) * 180.0f / PI;
}

bool calibrarGiroscopio() {
    float suma = 0.0f;

```

```

    LecturaIMU lectura;
    for (uint16_t i = 0; i < MUESTRAS_CALIBRACION; ++i) {
        if (!leerIMU(lectura)) return false;
        suma += lectura.gy;
        delay(PERIODO_SENSOR); // Solo en setup; motor apagado.
    }
    offsetGy = suma / MUESTRAS_CALIBRACION;
    return true;
}

void detenerMotor() {
    analogWrite(PIN_ENA, 0); // Quitar potencia primero.
    digitalWrite(PIN_IN1, LOW);
    digitalWrite(PIN_IN2, LOW);
    pwmActual = 0;
    pwmObjetivo = 0;
}

void aplicarMotor() {
    if (pwmActual == 0) {
        analogWrite(PIN_ENA, 0);
        digitalWrite(PIN_IN1, LOW);
        digitalWrite(PIN_IN2, LOW);
        return;
    }
    digitalWrite(PIN_IN1, pwmActual > 0 ? HIGH : LOW);
    digitalWrite(PIN_IN2, pwmActual > 0 ? LOW : HIGH);
    analogWrite(PIN_ENA, abs(pwmActual));
}

int calcularObjetivo(float angulo) {
    const float magnitud = fabsf(angulo);
    if (magnitud < ZONA_MUERTA) return 0;
    const float limitado = magnitud > ANGULO_MAXIMO ? ANGULO_MAXIMO : magnitud;
    const float proporcional = (limitado - ZONA_MUERTA) /
        (ANGULO_MAXIMO - ZONA_MUERTA);
    const int pwm = (int)(PWM_MINIMO + proporcional * (PWM_MAXIMO - PWM_MINIMO) + 0.5f);
    return angulo >= 0.0f ? pwm : -pwm;
}

void actualizarRampa() {
    if (estado != ACTIVO) {
        detenerMotor();
        return;
    }
    // Al invertir: primero llegar a cero. La direccion cambia en otro paso.
    int destino = pwmObjetivo;
    if ((pwmActual > 0 && destino < 0) || (pwmActual < 0 && destino > 0)) destino = 0;
    if (pwmActual < destino) {
        pwmActual += PASO_RAMPA;
        if (pwmActual > destino) pwmActual = destino;
    } else if (pwmActual > destino) {
        pwmActual -= PASO_RAMPA;
        if (pwmActual < destino) pwmActual = destino;
    }
    aplicarMotor();
}

void activarParo(const char *motivo) {
    detenerMotor(); // Sin rampa en una falla.
    if (estado == ACTIVO) {
        Serial.print("PARO DE SEGURIDAD: ");
        Serial.println(motivo);
    }
    estado = PARO;
    ultimoReintento = millis();
}

void actualizarSensor() {
    LecturaIMU lectura;
    if (!leerIMU(lectura)) {
        activarParo("fallo de lectura del MPU-6050");
        return;
    }
}

```

```

const uint32_t ahoraUs = micros();
const float dt = (uint32_t)(ahoraUs - ultimaLecturaUs) / 1000000.0f;
ultimaLecturaUs = ahoraUs;
if (dt <= 0.0f || dt > 0.10f) {
    activarParo("intervalo de control demasiado largo");
    return;
}
const float anguloAccel = pitchAcelerometro(lectura);
const float velocidadGy = SENTIDO_PITCH * (lectura.gy - offsetGy);
const float alfa = TAU_FILTRO / (TAU_FILTRO + dt);
pitch = alfa * (pitch + velocidadGy * dt) + (1.0f - alfa) * anguloAccel;
pwmObjetivo = calcularObjetivo(pitch);
}

void intentarRecuperacion(uint32_t ahora) {
    if (estado == PARO && (uint32_t)(ahora - ultimoReintento) >= PERIODO_REINTENTO) {
        ultimoReintento = ahora;
        Wire.end();
        iniciarBus();
        if (identificarIMU() && configurarIMU()) {
            estado = RECUPERANDO;
            inicioRecuperacion = millis();
        }
    } else if (estado == RECUPERANDO &&
        (uint32_t)(ahora - inicioRecuperacion) >= ESPERA_DESPERTAR) {
        LecturaIMU lectura;
        if (!leerIMU(lectura)) {
            activarParo("sensor aun no disponible");
            return;
        }
        // Conservar la calibracion; reiniciar angulo y rampa, sin integrar la pausa.
        pitch = pitchAcelerometro(lectura);
        detenerMotor();
        ultimaLecturaUs = micros();
        ultimoSensor = ultimoMotor = millis();
        primerReporte = true;
        ultimoCentro = -1;
        estado = ACTIVO;
        Serial.println("SENSOR RECUPERADO: control habilitado nuevamente.");
    }
}

void informarCambios() {
    if (estado != ACTIVO || !Serial) return;
    const float magnitud = fabsf(pitch);
    const int centro = magnitud <= ZONA_CENTRO ? 1 : 0;
    const int grado = (int)roundf(pitch);
    const int porcentaje = (abs(pwmActual) * 100 + PWM_MAXIMO / 2) / PWM_MAXIMO;
    const int sentidoMotor = (pwmActual > 0) - (pwmActual < 0);
    const int sentidoSensor = centro ? 0 : (pitch > 0.0f ? 1 : -1);
    const int intensidad = magnitud < 15.0f ? 0 : (magnitud < 30.0f ? 1 : 2);
    if (!primerReporte && grado == ultimoGrado && porcentaje == ultimoPorcentaje &&
        sentidoMotor == ultimoSentidoMotor && centro == ultimoCentro &&
        intensidad == ultimaIntensidad && sentidoSensor == ultimoSentidoSensor) return;
    if (centro != ultimoCentro) {
        if (centro) Serial.println("ENTRA AL CENTRO: sensor dentro de +/-2 grados.");
        else if (ultimoCentro == 1) Serial.println("SALE DEL CENTRO.");
    }
    Serial.print("Inclinacion: ");
    Serial.print(centro ? "centrado" : (sentidoSensor > 0 ? "adelante" : "atras"));
    Serial.print(" | ");
    Serial.print(grado);
    Serial.print(" grados | ");
    Serial.print(intensidad == 0 ? "leve" : (intensidad == 1 ? "moderada" : "fuerte"));
    Serial.print(" | Motor: ");
    Serial.print(sentidoMotor == 0 ? "detenido" : (sentidoMotor > 0 ? "adelante" : "reversa"));
    Serial.print(" | PWM: ");
    Serial.print(porcentaje);
    Serial.println(" %");
    ultimoGrado = grado;
    ultimoPorcentaje = porcentaje;
    ultimoSentidoMotor = sentidoMotor;
    ultimoCentro = centro;
    ultimaIntensidad = intensidad;
}

```

```

    ultimoSentidoSensor = sentidoSensor;
    primerReporte = false;
}

void actualizarMatriz() {
    uint8_t imagen[8][12] = {};
    if (estado != ACTIVO) {
        for (int fila = 0; fila < 8; ++fila) { // X: paro o fallo inicial.
            imagen[fila][fila + 2] = 1;
            imagen[fila][9 - fila] = 1;
        }
    } else if (fabsf(pitch) <= ZONA_CENTRO) {
        for (int fila = 0; fila < 8; ++fila)
            for (int columna = 0; columna < 12; ++columna)
                if (fila == 0 || fila == 7 || columna == 0 || columna == 11)
                    imagen[fila][columna] = 1;
    } else {
        const float limitado = constrain(pitch, -ANGULO_MAXIMO, ANGULO_MAXIMO);
        const int fila = (int)(3.0f - limitado * 3.0f / ANGULO_MAXIMO + 0.5f);
        imagen[fila][5] = imagen[fila][6] = 1;
        imagen[fila + 1][5] = imagen[fila + 1][6] = 1;
    }
    matriz.renderBitmap(imagen, 8, 12);
}

void setup() {
    // PRIMERA ACCION: fijar las salidas de motor a nivel bajo.
    digitalWrite(PIN_ENA, LOW);
    digitalWrite(PIN_IN1, LOW);
    digitalWrite(PIN_IN2, LOW);
    pinMode(PIN_ENA, OUTPUT);
    pinMode(PIN_IN1, OUTPUT);
    pinMode(PIN_IN2, OUTPUT);
    analogWriteResolution(8);
    detenerMotor();
    Serial.begin(115200);
    const uint32_t inicioSerie = millis();
    while (!Serial && (uint32_t)(millis() - inicioSerie) < 1500) delay(1);
    matriz.begin();
    iniciarBus();
    if (!identificarIMU() || !configurarIMU()) {
        Serial.println("ERROR INICIAL: MPU-6050 no responde 0x68 o no se configura.");
        Serial.println("Motor apagado. Revisar conexiones y pulsar RESET.");
        actualizarMatriz();
        return; // BLOQUEADO: no arranca con sensor malo.
    }
    delay(ESPERA_DESPERTAR);
    Serial.println("MPU-6050 detectado. Mantener sensor QUIETO durante 5 segundos.");
    if (!calibrarGiroscopio()) {
        Serial.println("ERROR INICIAL: fallo durante la calibracion. Pulsar RESET.");
        actualizarMatriz();
        return;
    }
    LecturaIMU lectura;
    if (!leerIMU(lectura)) {
        Serial.println("ERROR INICIAL: no se pudo obtener el angulo inicial.");
        actualizarMatriz();
        return;
    }
    pitch = pitchAcelerometro(lectura);
    ultimaLecturaUs = micros();
    ultimoSensor = ultimoMotor = ultimoSerie = ultimaMatriz = millis();
    estado = ACTIVO;
    Serial.print("Calibracion lista. Offset Y: ");
    Serial.print(offsetGy, 4);
    Serial.println(" grados/s. Sistema listo.");
    actualizarMatriz();
}

void loop() {
    const uint32_t ahora = millis();
    if (estado == ACTIVO && (uint32_t)(ahora - ultimoSensor) >= PERIODO_SENSOR) {
        ultimoSensor = ahora;
        actualizarSensor();
    }
}

```

```

}
if (estado == PARO || estado == RECUPERANDO) intentarRecuperacion(millis());
const uint32_t despues = millis();
if ((uint32_t)(despues - ultimoMotor) >= PERIODO_MOTOR) {
    ultimoMotor = despues;
    actualizarRampa();
}
if ((uint32_t)(despues - ultimoSerie) >= PERIODO_SERIE) {
    ultimoSerie = despues;
    informarCambios();
}
if ((uint32_t)(despues - ultimaMatriz) >= PERIODO_MATRIZ) {
    ultimaMatriz = despues;
    actualizarMatriz();
}
// Sin delay(): las cuatro tareas se programan con millis()/micros().
}

```